International Journal of Innovative Engineering Applications

Journal homepage: https://dergipark.org.tr/ijiea

ADAPTIVE HONEYPOT SYSTEMS VIA RAG: ENHANCING THREAT INTELLIGENCE WITH GENERATIVE AI

Anıl Sezgin*[1,2]

[1]*Research and Development, Siemens A.S., Istanbul 34870, Turkey*
[2]*Department of Computer Engineering, Istanbul Arel University, Istanbul 34537, Turkey*

Abstract

Original scientific paper

Honeypots have long been invaluable resources for intrusion detection and cyber threat intelligence, yet they suffer from an intractable trade-off: low-interaction systems are too artificial, and high-interaction systems pose operational risks and scalability challenges. This paper introduces a new honeypot architecture that uses Retrieval-Augmented Generation (RAG) with the Llama 3.1 8B model to overcome this fidelity–risk dilemma. Instead of running live commands, our system uses a curated database of sanitized command–output pairs for historic Linux commands. When an attacker issues a command, the highest-relevant historic output is recalled and contextualized via Large Language Model (LLM), and the response is empirically informed and dynamically flexible. This architecture maintains realistic interactions without allowing compromise. We built an extensible pipeline spanning data acquisition, preprocessing, retrieval, and response generation, complemented with logging for threat intelligence purposes. Evaluation was performed on six hundred canonical Linux commands using BLEU and ROUGE metrics. Analysis indicates that the RAG-enhanced variant is an order-of-magnitude improvement beyond vanilla LLM setup, with BLEU and ROUGE-L scores rising from 0.04 and 0.24, respectively, to 0.47 and 0.72, respectively. Beyond quantitative fidelity, qualitative analysis indicates that RAG strongly diminishes hallucinations, secures session consistency, and enhances attacker engagement. Extended and more coherent adversary sessions give the defender richer behavioral context with less compromise detection risk. This proposed system illustrates that generative AI, when tied to empirical basis, can achieve high-fidelity deception without operational exposure. Findings demonstrate not only the technical possibility of RAG-based honeypots but also their promise for use in scalable, adaptive, and safe deception resources for both research infrastructures and operational uses.

Keywords: *Large language models, threat intelligence, adaptive honeypot systems.*

RAG İLE UYARLANABİLİR HONEYPOT SİSTEMLERİ: ÜRETKEN YAPAY ZEKA İLE TEHDİT İSTİHBARATININ GÜÇLENDİRİLMESİ

Özet

Orijinal bilimsel makale

Honeypot'lar uzun süredir saldırı tespiti ve siber tehdit istihbaratı için vazgeçilmez kaynaklar olmuştur, ancak çözülmesi güç bir ikilemden muzdariptirler: düşük etkileşimli sistemler fazla yapay kalırken, yüksek etkileşimli sistemler operasyonel riskler ve ölçeklenebilirlik sorunları doğurmaktadır.. Bu makale, bu gerçekçilik–risk ikilemini aşmak için Llama 3.1 8B modeliyle birlikte Retrieval-Augmented Generation (RAG) kullanan yeni bir honeypot mimarisi sunmaktadır. Sistem, canlı komutları çalıştırmak yerine, Linux komutlarına ait temizlenmiş komut–çıktı çiftlerinden oluşan bir veritabanı kullanmaktadır. Bir saldırgan komut girdiğinde, en yüksek derecede alakalı geçmiş çıktı geri çağrılır, büyük dil modeli tarafından bağlama oturtulur, busayede yanıt hem deneysel olarak temellendirilmiş hem de dinamik olarak esnek olmaktadır. Bu mimari, sistemin ele geçirilmesine izin vermeden gerçekçi etkileşimleri sürdürebilmektedir. Veri edinimi, ön işleme, retrieval ve yanıt üretimini kapsayan, tehdit istihbaratı amaçlı loglama ile desteklenen genişletilebilir bir pipeline geliştirilmiştir. Değerlendirme, BLEU ve ROUGE metrikleri kullanılarak altı yüz temel Linux komutu üzerinde gerçekleştirilmiştir. Analiz, RAG ile güçlendirilmiş varyantın, vanilla LLM kurulumunun çok ötesinde, büyüklük mertebesinde bir iyileşme sağladığını göstermektedir, BLEU ve ROUGE-L skorları sırasıyla 0,04 ve 0,24'ten 0,47 ve 0,72'ye yükselmiştir. Nicel gerçekçiliğin ötesinde, nitel analiz RAG'in halüsinasyonları büyük ölçüde azalttığını, oturum tutarlılığını güvence altına aldığını ve saldırgan etkileşimini artırdığını göstermektedir. Daha uzun ve daha tutarlı adversary oturumları, savunmacıya daha az tespit riskiyle daha zengin davranışsal bağlam sunmaktadır. Önerilen bu sistem, üretken yapay zekanın, deneysel temele dayandırıldığında, operasyonel riske maruz kalmadan yüksek doğruluklu aldatma sağlayabileceğini ortaya koymaktadır. Bulgular, RAG tabanlı honeypotların teknik olarak mümkün olduğunu, aynı zamanda araştırma altyapıları ve operasyonel kullanımlar için ölçeklenebilir, uyarlanabilir ve güvenli aldatma kaynakları olarak umut vadettiğini göstermektedir.

Anahtar Kelimeler: *Büyük dil modelleri, tehdit istihbaratı, uyarlanabilir honeypot sistemleri.*

*Corresponding author.
E-mail address: anil.sezgin@siemens.com (A. Sezgin)
Received 06 October 2025; Received in revised form 05 November 2025; Accepted 06 November 2025
2587-1943 | © 2025 IJIEA. All rights reserved. Doi: https://doi.org/10.46460/ijiea.1797978

1 Introduction

The evolutions of cyber threats have made it necessary for both researchers and practitioners to continually assess the efficacy of traditional defense methods. Honeypots, in particular, artificial systems specifically designed to lure and analyze malicious actors, have long been prized as critical tools for intrusion detection, gathering of threat intelligence, and understanding of adversary behaviors. Their role in today's cybersecurity landscape is bivalent: first, they provide defender with critical information regarding novel methods of attack, variants of malware, and exploit tactics; second, they serve as active deception mechanisms that rob attackers of resources and reduce the potential for successful breaches in operational settings. Despite their long-acknowledged merits, honeypots face significant challenges, particularly in terms of balancing the realism, scalability, and safety of operation. The trade-off of interaction fidelity and associated security threats due to exposure of an environment with high realism is one of the longest-running and chronic shortcomings of traditional honeypot solutions, according to [1] and [2].

Low-interaction honeypots are specifically crafted to emulate only a limited set of services or capabilities. Their simple nature makes it easy to deploy, minimizes maintenance, and pose little risk of being compromised. Their simplicity, however, makes them vulnerable to discovery, thus limiting their potential to record significant attacker behaviors. As shown in [3], lightweight solutions present cost-effective protection mechanisms, yet they may lack the complexity that would make it hard for serious attackers to deal with them effectively. High-interaction honeypots, on the other hand, implement systems that provide attackers with an entire working environment, in some cases, using real operating systems and applications. These provide voluminous data amenable to academic research but equally increase the risk of compromise, for instance, as highlighted in [4]. Their administration overhead and associated security risks limit their use to special cases such as research settings.

The bifurcation of low and high interaction honeypots has led to a sustained gap in Cybersecurity defense mechanisms. There is a need for an amalgam that combines the authenticity of high-interaction honeypots with the scalability and security typical of low-interaction methods. Research such as [5] depicts the rising relevance of adaptive deception architectures alongside the issues in building secure systems in the presence of constant adversary threats. Moreover, [6] emphasizes that with the inclusion of machine learning, scalability and adaptability remain major issues.

Recent advances in the field of natural language processing create new possibilities for addressing this challenge. Large Language Models have the potential to create contextually pertinent responses, and Retrieval-Augmented Generation models ensure that such responses have empirical evidence grounding them. Within the context of honeypot creation, this means an approach shift: rather than actually running commands in working systems, a honeypot would pull historical command-output pairs from a knowledge store and use an LLM to synthesize dynamic and realistic responses. As shown in [7], large language models can effectively keep attackers engaged through credible and context-aware deception. Likewise, [8] demonstrates how LLMs could automate intelligence generation derived from interactions with attackers. Together, these works suggest that generative AI could revolutionize honeypot architecture through the promise of adaptivity and authenticity without subjecting live systems to exposure to vulns.

By grounding outputs in actual command-line logs, our method maintains authenticity while granting adaptive variation. Through this tension between empirical foundation and generative variability, the system creates a believable illusion of a live environment despite even skilled attackers. Further, because attackers never engage with manipulable processes, but rather only simulated outputs, the architecture inherently minimizes risk. This scalability of such a system makes it relevant for organizations ranging in size from small research labs to SMEs, thus widening the availability of more advanced deception techniques.

The novelty and contribution of the present work may be stated briefly thus:

•High-fidelity, low-risk terminal honeypot: Here, we unveil a RAG-supported architecture that efficiently spoofs command-line interfaces without executing live processes, and end the decades-long fidelity-risk trade-off in honeypot architecture.

•APT-as-a-service and adaptive deception: By grounding response in real command-line logs and LLM-assisted contextual adaptation, our approach generates realistic outputs that shift in real-time along with attacker input, rendering deception more robust for modern adversaries.

The rest of this paper is thus structured: Section 2 summarizes the related work, albeit focusing on improvements in honeypot technologies, deception techniques, and recent applications of machine learning and big language models. Section 3 outlines the devised methodology, detailing our RAG-based terminal honeypot's architecture and the mechanisms that make it both faithful and safe. Section 4 illustrates the experimental setup and outcomes, in which we assess the system's realism, scalability, and robustness versus advanced adversary probing. Lastly, Section 5 concludes the paper and comments on potential future research, accenting the wider implications of adaptive, data-driven deception for the modern world of cybersecurity.

2 Related Work

Cybersecurity, intrusion detection, and honeypot-related defense have been burgeoning areas of research, with varying methods ranging from IoT to SDN, UAVs, critical infrastructures, and LLM-related systems. This section summarizes studies that together show the progression of the subject, indicate shortcomings, and indicate potential ways to develop intelligent, adaptive, and scalable defense mechanisms.

2.1 Honeypots and Deception Technologies

Traditional honeypots have been reimagined to increase their robustness in the face of modern threats. An in-depth analysis in relation to the use of machine learning in Internet of Things honeypots demonstrated that supervised and reinforcement learning models significantly increase detection levels while limiting false positives, despite prevalent computing challenges [6]. Further, a case comparison involving seven honeypot solutions highlighted the importance of effortless intergration with organizational infrastructures and identified Dionaea and Cowrie as flexible tools [1]. Specialized honeypots have also been developed: ShadowKube enhances Kubernetes security by converting compromised pods in real-time to honeypots [5], while PLC-specific metrics have been proposed to address certain operational technology challenges [4]. Similarly, TwinFedPot combines honeypots and digital twins along with federated distillation for intelligent traffic system protection [9].

These lightweight designs aim for SMEs, such as HoneyLite, which strikes a perfect balance between cost and detection effectiveness [3]. Some others concentrate on web attacks, with honeypots actually capturing malicious payloads and exchanging intelligence with MISP [10]. Venturing into new realms, MV-Honeypot showed that avatars could serve as decoys in metaverse environments in order to harvest threat intelligence [11]. Honeypots have even been used in database security, in which MySQL proved the highest targeted DBMS [12]. Furthering in its scope, a wider survey of deception techniques also mapped mathematical models for improving honeypot performance [2].

2.2 AI- and ML-Driven Cyber Defense

Machine learning and artificial intelligence methods increasingly characterize intrusion detection. Isolation Forest model-based honeypot-driven IDS made significant performance improvements using kernel-level data [13]. Kernel-level network threat management was enhanced through high-interactivity honeypot framework and detailed behavioral session capturing [14]. Cybersecurity prediction has been tackled through CyberGuard-X, which combines deep learning and anomaly detection for predicting precursors to attacks with high precision [15]. Likewise, AI-based analysis of honeypot traffic using LLMs allows for real-time anomaly detection in critical infrastructures [8].

Adaptive reinforcement learning has been used to maximize defense mechanisms for critical systems [16]. Game-theoretic methods in combination with honeypots were introduced to defeat UAV DoS attacks with minimal latency [17]. Graph embedding methods in association with RL improved decision-making for autonomous cyber defense [18]. For zero-day detection, Adaptive SAMKNN exhibited robust performance in IoT and IIoT settings [19]. Logistic regression on honeypot web logs was also effective in recognizing web attack behaviors [20].

Malware detection techniques that use few-shot learning, such as MalFSLDF, deal with detection when just small sets of samples would be made available [21]. Similarly, special honeypots for IoT hardware Trojans had been proposed [22]. H-DOCTOR was an integration of fuzzy inference engines and Docker-honeynets for runtime firewall adjustment for ransomware [23].

2.3 Large Language Models and Cybersecurity

The emergence of LLMs has created new paradigms in cybersecurity. Near-term applications of ChatGPT were validated in areas like threat hunting and access management, though concerns about hallucinations remain [24]. GenDFIR leveraged retrieval-augmented generation with Llama models to automate DFIR timeline analysis [25]. Natural language querying of drone data was achieved with a RAG-based interface, bridging user intent with complex IoD APIs [26]. Asset discovery in critical infrastructures benefited from LLMs' ability to synthesize heterogeneous data sources [27].

LLMs also trigger deception mechanisms: DecoyPot, an LLM-managed web API honeypot, generates realistic outputs to deceive attackers [7]. Reviews have synthesized these advancements, with qualitative analyses of applications of LLMs, vulnerabilities, and protection [28], and evaluations of LLMs in critical infrastructure protection that emphasize benchmarks, agentic AI, and deployment problems [29].

2.4 Software-Defined Networking and Moving Target Defense

SDN and moving target defense (MTD) architectures present additional innovation. DDoS detection and mitigation technology surveys in SDN settings grouped solutions by working level and performance [30]. Combining honeypots and stochastic game theory improved cost-effective MTD techniques [31]. SMASH continued this path by integrating SDN and MTD in the form of a honeypot-as-a-service model, essentially neutralizing insider threats [32]. HoneyTwin proposed an SDN edge-cloud hybrid architecture for IoT device protection in smart cities via ML-enabled honeypots [33]. VANETs were also tackled using ML-boosted honeypots examining attack rates for anticipating unknown threats [34].

2.5 Emerging Domains and Challenges

The research is now being expanded to cloud-native and microservice environments. Vulnerability-oriented trust was proposed for dynamic risk management in service mesh deployments [35]. Separately, these initiatives stress the path toward more adaptive, autonomous, and intelligent cybersecurity solutions using deception, ML, and LLMs in diverse application arenas.

3 Methodology

Architecture for the honeypot system is modular, promoting authenticity, flexibility, and security. Instead of running commands on an active operating system, the system mimics terminal interactions in the form of a carefully selected and curated knowledge repository and Retrieval-Augmented Generation (RAG) architecture

using the Llama 3.1 8B model. This section highlights the key elements of the methodology in the form of data acquisition and preprocessing, the RAG-based generation pipeline, logging and intel extraction, and the evaluation methodology.

The overall workflow is depicted in Figure 1. It begins with the capture of standard outputs from a real Linux system, which are then preprocessed and normalized into a structured JSON knowledge base. This knowledge base serves as the foundation for retrieval operations. When an attacker issues a command to the honeypot, the system retrieves the most relevant command–output pair from the knowledge base and feeds it to the LLM, which generates a contextually consistent response. The interaction is logged for later analysis, and the generated output is compared against the ground-truth actual output using automatic evaluation metrics. The internal workflow of the RAG pipeline is illustrated in Figure 2, detailing how retrieval, generation, and logging components interact within the system.

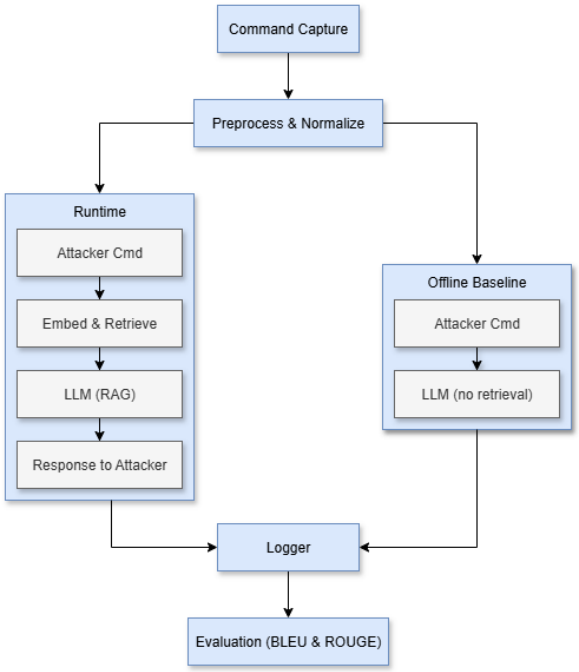


Figure 1. System architecture diagram.

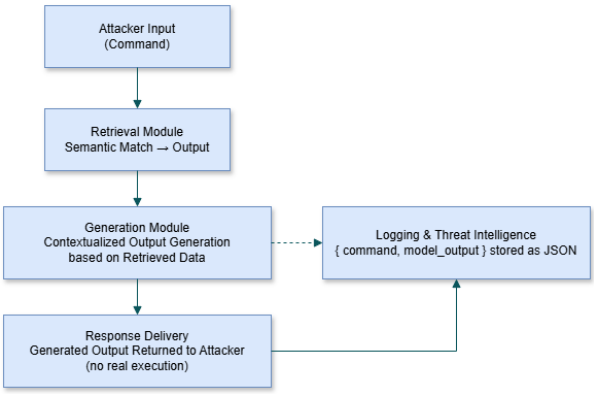


Figure 2. RAG pipeline with retrieval and logging flow.

To preserve authenticity, we first built a command capture module that would record outputs produced by typical Linux utilities like ls, ps, netstat, ifconfig, and cat. These command outputs form the basis of the original dataset used in building the knowledge base. For all that, the raw outputs in many cases contain system-dependent data elements, like process IDs, dates, or hostnames, that vary from one execution to another and may reveal inconsistencies if not handled appropriately.

A preprocessing module was thus introduced to cleanse and normalize the obtained outputs. This process involves removal of transient information (e.g., dynamic PIDs), formatting normalization from multiple systems, and anonymization of sensitive information. The cleaned data are thus aggregated in JSON objects that have two properties: command and its actual corresponding output. This organization ensures compatibility with the acquisition pipeline and is precise in terms of natural system behavior. An example entry from the constructed knowledge base is provided in Table 1.

Table 1. Example entries from the JSON-based command–output knowledge base.

Command	Actual Output
pwd	/home/user
whoami	root

The heart of the honeypot configuration is a Retrieval-Augmented Generation pipeline that is operating with the Llama 3.1 8B model. When an attacker is in contact with the honeypot and is feeding it a command, the system undergoes three essential processes.

The retrieval phase places the attacker's input command within a semantic embedding framework and selects the most similar historical command from the knowledge repository using a nearest-neighbor search, the corresponding actual output is then selected as the retrieval context. At the generation phase, the retrieved command–output pair is provided to the Llama 3.1 8B model, which produces a response corresponding to the recovered empirical context. This conditioning preserves realism while allowing for natural variations that help the honeypot remain undetected. Finally, at the response-delivery phase, the generated output is returned to the attacker, at no time is the command issued on an active system, thus eliminating the risk of compromise. As a consequence, this hybrid structure leverages the empirical efficacy of cataloged Linux outputs along with the flexibility of generative modeling, enabling long-running convincing interactions without disclosing any exploitable services.

For every attacker interaction, the honeypot maintains a detailed log that records both the user's command and the model's generated response. The logging module produces entries in JSON format with two key fields: command and model_output. These logs not only support system evaluation but also serve as valuable sources of threat intelligence, enabling defenders to analyze attacker behavior and refine the knowledge base over time. An example of such a log entry is shown in Table 2.

Table 2. Example of a honeypot log entry capturing attacker input and model response.

Command	Actual Output
hostname	server01
uname -r	5.15.0-70-generic
pwd	/home/user

To assess the fidelity of the generated responses, we compare the model output with the actual Linux output retrieved from the knowledge base. The comparison is performed using two well-established natural language generation metrics: BLEU and ROUGE.

The BLEU score measures the overlap of n-grams between the generated output and the reference output, placing emphasis on precision. It is formally defined as in (1).

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right) \quad (1)$$

where p_n is the modified n-gram precision, w_n is the weight assigned to each n-gram length, and *BP* is the brevity penalty to discourage overly short outputs.

The ROUGE metric complements BLEU by focusing on recall, measuring the proportion of n-grams in the reference that are successfully captured in the generated output. The most common variant, ROUGE-N, is defined as in (2).

$$\text{ROUGE-N} = \frac{\sum_{S\in\{\text{Ref}\}} \sum_{gram_n \in S} \text{Count}_{match}(gram_n)}{\sum_{S\in\{\text{Ref}\}} \sum_{gram_n \in S} \text{Count}(gram_n)} \quad (2)$$

where Count_{match} denotes the number of overlapping n-grams between the generated and reference outputs.

Together, BLEU and ROUGE provide a balanced evaluation of the honeypot's ability to generate responses that are both precise and comprehensive in reproducing realistic terminal outputs. High scores on these metrics indicate that the honeypot achieves its central objective: presenting attackers with convincing system behavior that is indistinguishable from a live environment.

4 Experimental Setup and Results

The knowledge base used for evaluation was produced by our dedicated command capture module. Using this module, we executed and recorded outputs for six hundred distinct Linux commands in representative environments. Each raw terminal output was then captured, sanitized, and converted into a structured JSON record of the form *{"command": <command>, "actual_output": <output>}*. The command capture module automates consistent invocation and logging of commands, ensuring that captured outputs preserve realistic formatting and distributional properties of real systems while providing a reproducible input stream for preprocessing, retrieval indexing, and downstream evaluation. This dataset constitutes the empirical foundation for all retrieval operations and provides the ground truth against which generated outputs are compared.

Following capture, the raw outputs underwent a deterministic preprocessing pipeline designed to remove or normalize highly volatile tokens while preserving structural fidelity. Volatile elements such as timestamps, ephemeral process identifiers, memory addresses, MAC addresses, and transient counters were replaced with deterministic placeholders; column alignments and spacing conventions from command utilities like ps and ls were standardized; and any sensitive or host-specific tokens were anonymized. These steps reduce spurious mismatches during automatic scoring and prevent trivial detection cues that could compromise the deception. The cleaned command–output pairs were serialized into a compact JSON schema with two primary fields: command and actual_output. This canonical form allows efficient building of embedding indices for nearest-neighbor retrieval and guarantees that all downstream evaluations reference the same empirical content.

The experimental comparison was framed as a controlled contrast between two model variants. In the first variant, referred to as the vanilla configuration, the input command was presented directly to the generative model (Llama 3.1 8B) without retrieval augmentation, relying solely on the model's internal knowledge and generative capacity. This variant simulates a naive LLM-powered honeypot that produces outputs based only on learned distributions. In the second variant, referred to as the RAG configuration, the input command was embedded and matched to the most similar command entry in the JSON knowledge base; the corresponding actual_output was then provided to the generative model as grounding context, and the model produced a response conditioned on that retrieved empirical output. Both variants shared identical decoding parameters, temperature settings, and tokenization schemes to ensure a fair comparison where retrieval was the only differing factor.

For each of the six hundred canonical commands, both variants generated outputs which were logged together with the reference actual_output. Interaction logs therefore contained triplets of the form *{"command": ..., "actual_output": ..., "model_output": ...}* for the RAG condition and analogous entries for the vanilla condition. These logs served as the basis for automated scoring and for qualitative inspection of multi-step sequences. To quantify fidelity, we employed BLEU and ROUGE metrics, computed per command and averaged across the entire corpus. BLEU was computed as cumulative n-gram precision up to 4-grams with the standard brevity penalty; ROUGE-1 and ROUGE-2 measured unigram and bigram recall-like overlap, and ROUGE-L used the longest common subsequence to capture sequence-level alignment. These metrics were chosen because they capture complementary aspects of lexical and structural similarity between generated outputs and empirical ground truth. Table 3 summarizes the averaged evaluation results across all six hundred command instances for both configurations.

Table 3. Comparative evaluation of RAG-augmented generation versus vanilla generation, averaged over six hundred canonical commands.

Model Variant	Avg BLEU	Avg ROUGE-1	Avg ROUGE-2	Avg ROUGE-L
With RAG	0.471266	0.725436	0.630274	0.721736
Without RAG	0.0436022	0.265539	0.174637	0.244430

The empirical differences are stark and consistent across metrics. The RAG-augmented configuration attains an average BLEU near 0.47, in contrast to the vanilla

model's BLEU of approximately 0.04. ROUGE metrics further corroborate these findings, showing large absolute improvements (0.45–0.48) and confirming better n-gram coverage and sequence alignment. ROUGE metrics corroborate the BLEU findings: ROUGE-1, ROUGE-2, and ROUGE-L all show large absolute improvements (on the order of 0.45–0.48), denoting superior unigram coverage, phrase-level fidelity, and sequence alignment when retrieval is used as grounding.

BLEU and ROUGE scores work well in measuring linguistic similarities, but their use in this research was driven by their ability to be easily interpreted and reproduced in generative models of natural language. Nevertheless, it is crucial to highlight that scores calculated through metrics such as BLEU and ROUGE do not fully represent behavioral realism or deceive efficacy. Linguistic similarities do not possess an inherent ability to represent attacker honeypot interactions in a realistic or engaging manner.

The quantitative comparison between the retrieval-augmented model and the baseline vanilla model is presented in Figure 3. These results indicate that retrieval augmentation not only reduces hallucinations but also enables the system to maintain more realistic and coherent terminal responses, making it significantly more effective as a high-fidelity honeypot. Nevertheless, it is worth considering that BLEU and ROUGE scores are essentially measures of overlap that do not necessarily well represent semantic fidelity and realism in terms of deceptive generations. Although higher scores of such metrics imply higher linguistic similarities, they do not take cognizance of deceptive engagement qualities in terms of responses that are essentially intended to deceive or manipulate in a communicative interaction.

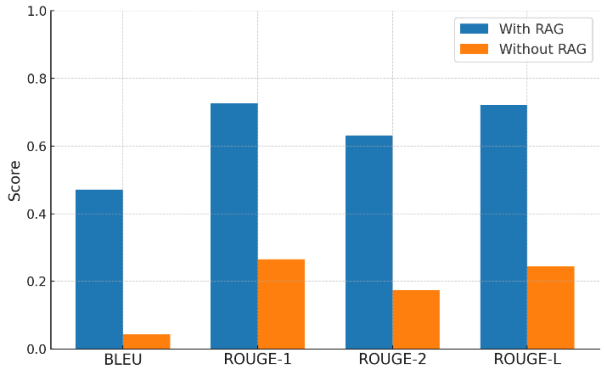


Figure 3. Performance comparison of RAG and baseline models.

Beyond aggregate scores, qualitative analysis of individual logs reveals the practical effects of retrieval augmentation. Under the vanilla condition, the model frequently produced fluent but technically incorrect outputs, listing files that are absent in the canonical actual_output, inventing service names or ports, or producing atypical column spacing and formatting inconsistent with *ps* and *ls* outputs. Such hallucinations, while superficially plausible, provide salient detection cues for an adversary performing cross-checks. By contrast, the retrieval-augmented model constrains its generation to the distribution represented in the retrieved actual_output, producing outputs that match expected permission strings, user/group names, process columns, and numeric fields. This constraint yields a marked reduction in invented entities and formatting anomalies.

Session consistency is another dimension where the RAG approach shows clear advantages. Real attackers rarely issue a single isolated command; instead, they probe systems with sequences of commands to validate findings and escalate privileges. In multi-step sequences such as listing a directory and then attempting to cat a file from that listing, the retrieval-augmented system preserved coherence: filenames that appeared in a prior ls output could be subsequently referenced with plausible content or access denial messages consistent with the captured environment. The vanilla model, deprived of grounding context, often failed to maintain these invariants, generating contradictions that would likely expose the deception after only a few queries.

Operational performance considerations were also examined because retrieval introduces computational overhead. The primary cost is the nearest-neighbor search in embedding space; for our experiment the index size was small (six hundred canonical entries) and retrieval latency remained modest, on the order of tens of milliseconds per query. End-to-end response times therefore remained comfortably within interactive limits and the added latency is negligible compared to the fidelity gains. For larger deployments, engineering choices such as approximate nearest neighbor indices or hierarchical retrieval can preserve low-latency behavior while scaling to large knowledge bases.

The logging produced by the system also yields secondary benefits in the form of enriched threat intelligence. Because the RAG-enabled honeypot produces credible outputs that sustain engagement, captured command sequences tend to be longer and more revealing. Analysis of these longer sessions provides richer data on reconnaissance patterns and attacker behavior, enabling defenders to correlate attempted commands with known Tactics, Techniques, and Procedures (TTPs), to identify novel probing sequences, and to prioritize detection controls. Even though no commands are executed on a live system, the command sequences themselves are useful telemetry for defenders. When combined with high-fidelity generated outputs, these logs offer a safe yet informative means to study adversarial tactics.

5 Discussion and Future Work

Experiment's results demonstrate the critical role that retrieval-augmented generation can take in constructing today's honeypots. By basing the outputs of language models in an expertly selected set of actual command–output pairs, our approach makes progress toward one of the oldest and greatest deception tech challenges: realism with minimal risk. Our subsequent discussion situates our results in the wider research space, examines operational implications, and sketches potential points for future work.

A key takeaway from the experiments conducted is the potential for RAG-based honeypots to achieve a satisfactory balancing act of faithfulness and safety. Traditional low-interaction solutions often have difficulty convincingly emulating real-world settings, while high-

interaction solutions, even with their increased level of realism, risk subjecting defenders to potential breaches themselves. The results show that retrieval augmentation is a competitive compromise. The BLEU and ROUGE scores support that the generated response through retrieval is significantly more like actual outputs, and thus more likely to keep attackers engaged. This balancing act is of particular value in deployments where organizations have limited resources or would not wish to host live vulnerable systems, yet would still desire the capability to observe adversary activities.

A further significant contribution is found in the mitigation of hallucinations and inconsistencies, which are prevalent challenges linked to large language models. The baseline model employed in this context often generated outputs that appeared plausible yet were technically inaccurate. Conversely, the system enhanced by RAG was limited by the knowledge base, resulting in consistent and verifiable outcomes. This enhancement not only bolsters the credibility of the honeypot but also reduces the likelihood that attackers will readily recognize the system as a ruse. Maintaining attacker engagement is crucial, given that extended sessions yield more comprehensive behavioral data and facilitate deeper insights into tactics and techniques.

The implications stretch beyond simple deception. Since attackers are more likely to persevere in their operations within an environment of deception, these generated logs translate to higher-quality threat intelligence. The entire command sequences, tool deployments, and behavioral trends that the honeypot captures can also complement broader defensive plans. By integrating this data with intrusion detection systems or cyber threat feeds, it would better enable defenders to anticipate adversary moves and devise proactive counterplays. RAG-based honeypots, in turn, can serve not only as entrapments but also valuable sensors in the larger picture of security.

At the same time, the current evaluation setup reveals important limitations. BLEU and ROUGE, while convenient and interpretable, measure surface-level textual overlap and cannot fully capture semantic realism or contextual continuity across long sessions. Although the metrics showed dramatic improvements when retrieval was applied, they cannot replace human judgment in assessing deception strength. Future work should therefore incorporate structured evaluations involving penetration testers or red teams, offering qualitative perspectives on how well the system withstands real adversarial scrutiny. This step will be essential for validating the operational effectiveness of the approach.

Scalability is a major challenge for future deployments. The prototype knowledge base that we built was assembled from six hundred typical Linux commands, picked off by a command capture module. While sufficient for experimental verification, real operational environments demand broader coverage. Increasing the knowledge base to cover a variety of commands, operating systems, and application scenarios will be critical in ensuring robustness to skilled adversaries. Mechanisms for the programmatic, periodic capture, preprocessing, and incorporation of new command-output pairs will enable agility, and efficient retrieval mechanisms will be critical in maintaining low-latency response at scale.

Also at stake is adversarial robustness. With generative models becoming more prominent in deception, attackers will seek to fingerprint them or exploit vulnerabilities in retrieval mechanisms. Resilience to probing or evasion attacks will necessitate defenses in the spirit of adversarial machine learning. Randomized response generation, adaptive retrieval, or embeddings resistant to perturbation could all serve to thwart detection. Solving this problem will prove essential in ensuring the long-haul efficacy of generative honeypots in contested spaces.

In addition to its experimental findings, this proposed honeypot system based on RAG also has immediate applicability in real-world defense systems. In a Security Operations Center (SOC) setting, it is possible to use it as a decoy system that inputs attacker engagement data directly into a centralized monitoring system in use, such as a SIEM or a SOAR system. This allows real-time analysis of attacker engagement data from honeypots and real-time network activities, enabling prioritized incident response. In incident responses, for instance, analyzing the logs created by an automated system such as RAG to generate JSON-formatted data directly assists digital forensic systems in recreating attacker activities without engaging assets in real networks. Therefore, this system connects research and defense actions, enabling actionable research findings from honeypots in real cybersecurity defense systems in enterprises.

In comparison to current honeypot systems such as Cowrie, Dionaea, and HoneyLite, our proposed architecture has a unique strength in its ability to adaptively interact through an RAG system. Current honeypots are only capable of reactive interactions through scripts or replayed conversations that lack realism when attackers behave in an unconventional manner compared to scripts. In contrast, our system is capable of generating responses through retrieval-augmented generation that are well-grounded in context while at the same time varying in a manner that is secure as it does not pose a greater threat to our system.

Despite its efficacy in protecting against system attacks through interaction isolation, it is crucial to take heed of threats that specifically target an environment involving an LLM. Examples of such threats include prompt attacks, whereby an attacker uses a prompt to influence an interaction, or fingerprinting, whereby an attacker tries to determine the type of model or its parameters. By including defenses against such threats in its system model, it will improve its overall robustness and integrity.

One of the most important aspects of generative deception systems is related to their ethical use. In this respect, it should be kept in mind that such systems are designed to mimic interactions with attackers. This means that such systems should be designed in a manner that prevents the spread of misleading or deceptive information as a result of generated interactions. This will require ensuring that such systems are contained in a setting that prevents interactions with attackers from affecting real users. In research use, issues of transparency

and gaining informed consent will also be paramount in such an endeavor. Information acquisition, even for honeypots, involves questions of legality and privacy that vary from place to place. Wary use will involve establishing transparent systems that align defense imperatives and compliance requirements. Meanwhile, provisions should also exist to prevent unintentional generative errors from distributing sensitive content or producing false results accidentally. Accordingly, establishing installations for the best practices in responsible use of retrieval-augmented deception technology will form part of future research endeavors. Future research will also incorporate human evaluation and attacker engagement metrics to provide a more comprehensive measure of behavioral realism and deception effectiveness.

Acknowledgements

We would like to express our sincere gratitude to Siemens A.S. for their valuable support throughout the completion of this study.

Declaration

Ethics committee approval is not required.

References

[1] Morić, Z., Dakić, V., & Regvart, D. (2025). Advancing Cybersecurity with Honeypots and Deception Strategies. *Informatics*, 12, 1-37.

[2] Javadpour, A., Ja'fari, F., Taleb, T., Shojafar, M., & Benzaïd, C. (2024). A comprehensive survey on cyber deception techniques to improve honeypot performance. *Computers & Security*, 140, 1-39.

[3] AlQahtan, N., AlOlayan, A., AlAjaji, A., & Almaslukh, A. (2025). HoneyLite: A Lightweight Honeypot Security Solution for SMEs. *Sensors*, 25, 1-17.

[4] Heluany, J. B. (2024). PLC Honeypots: Enhancing Interaction-Level Assessment. *Electronics*, 13, 1-18.

[5] Chen, Q., Liu, Y., Tan, R., Jin, Z., Xiao, J., Wang, X., Zhang, F., & Liu, Q. (2025). Shadowkube: Enhancing Kubernetes security with behavioral monitoring and honeypot integration. *Cybersecurity*, 8, 1-19.

[6] Lanz, S., Pignol, S. L., Schmitt, P., Wang, H., Papaioannou, M., Choudhary, G., & Dragoni, N. (2025). Optimizing Internet of Things Honeypots with Machine Learning: A Review. *Applied Sciences*, 15, 1-30.

[7] Sezgin, A., & Boyacı, A. (2025). DecoyPot: A large language model-driven web API honeypot for realistic attacker engagement. *Computers & Security*, 154, 1-9.

[8] Lanka, P., Gupta, K., & Varol, C. (2024). Intelligent Threat Detection—AI-Driven Analysis of Honeypot Data to Counter Cyber Threats. *Electronics*, 13, 1-28.

[9] Sahraoui, Y., Hadjkouider, A. M., Kerrache, C. A., & Calafate, C. T. (2025). TwinFedPot: Honeypot Intelligence Distillation into Digital Twin for Persistent Smart Traffic Security. *Sensors*, 25, 1-16.

[10] Visalom, R., Mihăilescu, M., Rughiniș, R., & Țurcanu, D. (2025). Intercepting and Monitoring Potentially Malicious Payloads with Web Honeypots. *Future Internet*, 17, 1-32.

[11] Sarang, A. D., Alawami, M. A., & Park, K. (2024). MV-Honeypot: Security Threat Analysis by Deploying Avatar as a Honeypot in COTS Metaverse Platforms. *CMES - Computer Modeling in Engineering and Sciences*, 141(1), 655-669.

[12] Rocha, A., & Pereira, F. S. F. (2025). What is the most targeted database? Using Honeypots to Characterize Attacks on Relational Databases. *Procedia Computer Science*, 256, 114-121.

[13] Alatawi, E., & Albalawi, U. (2025). Harnessing AI for Cyber Defense: Honeypot-Driven Intrusion Detection Systems. *Symmetry*, 17, 1-15.

[14] Yang, X., Yuan, J., Yang, H., Kong, Y., Zhang, H., & Zhao, J. (2023). A Highly Interactive Honeypot-Based Approach to Network Threat Management. *Future Internet*, 15, 1-31.

[15] Al Abdulwahid, A. (2025). AI-Driven Identification of Attack Precursors: A Machine Learning Approach to Predictive Cybersecurity. *Computers, Materials and Continua*, 85(1), 1751-1777.

[16] Yu, X., He, L., Geng, J., Liang, Z., Gan, Z., & Zhao, H. (2025). Dynamic Defense Strategy Selection Through Reinforcement Learning in Heterogeneous Redundancy Systems for Critical Data Protection. *Applied Sciences*, 15, 1-20.

[17] Miao, S., Li, Y., & Pan, Q. (2023). Honeypot Game Theory against DoS Attack in UAV Cyber. *Computers, Materials and Continua*, 76(3), 2745-2762.

[18] Wang, Z., Wang, Y., Xiong, X., Ren, Q., & Huang, J. (2025). A Novel Framework for Enhancing Decision-Making in Autonomous Cyber Defense Through Graph Embedding. Entropy, 27, 1-24.

[19] Agbedanu, P. R., Yang, S. J., Musabe, R., Gatare, I., & Rwigema, J. (2025). A Scalable Approach to Internet of Things and Industrial Internet of Things Security: Evaluating Adaptive Self-Adjusting Memory K-Nearest Neighbor for Zero-Day Attack Detection. *Sensors*, 25, 1-35.

[20] Wu, Q., Wen, S., Li, F., Liu, B., & Zhong, W. (2024). Web Attack Detection Based on Honeypots and Logistic Regression Algorithm. *Journal of Electrical Systems*, 20(3), 814-822.

[21] Guo, W., Xue, J., Lin, Y., Du, W., Hu, J., Shi, N., & Han, W. (2025). MalFSLDF: A Few-Shot Learning-Based Malware Family Detection Framework. *International Journal of Intelligent Systems*, 2025(1), 1-20.

[22] Omar, A. H. E., Soubra, H., Moulla, D. K., & Abran, A. (2024). An Innovative Honeypot Architecture for Detecting and Mitigating Hardware Trojans in IoT Devices. *IoT*, 5, 730-755.

[23] Amal, M. R., & Venkadesh, P. (2023). H-DOCTOR: Honeypot based firewall tuning for attack prevention. Measurement: *Sensors*, 25, 1-7.

[24] DeCusatis, C., Tomo, R., Singh, A., Khoury, E., & Masone, A. (2025). Cybersecurity Applications of Near-Term Large Language Models. *Electronics*, 14, 1-14.

[25] Loumachi, F. Y., Ghanem, M. C., & Ferrag, M. A. (2025). Advancing Cyber Incident Timeline Analysis Through Retrieval-Augmented Generation and Large Language Models. *Computers*, 14, 1-42.

[26] Sezgin, A. (2025). Natural Language Interfaces for Structured Query Generation in IoD Platforms. *Drones*, 9, 1-20.

[27] Coppolino, L., Iannaccone, A., Nardone, R., & Petruolo, A. (2025). Asset Discovery in Critical Infrastructures: An LLM-Based Approach. *Electronics*, 14, 1-24.

[28] Jaffal, N. O., Alkhanafseh, M., & Mohaisen, D. (2025). Large Language Models in Cybersecurity: A Survey of Applications, Vulnerabilities, and Defense Techniques. *AI*, 6, 1-45.

[29] Yigit, Y., Ferrag, M. A., Ghanem, M. C., Sarker, I. H., Maglaras, L. A., Chrysoulas, C., Moradpoor, N., Tihanyi, N., & Janicke, H. (2025). Generative AI and LLMs for Critical Infrastructure Protection: Evaluation Benchmarks, Agentic AI, Challenges, and Opportunities. *Sensors*, 25, 1-40.

[30] Su, Y., Xiong, D., Qian, K., & Wang, Y. (2024). A Comprehensive Survey of Distributed Denial of Service Detection and Mitigation Technologies in Software-Defined Network. *Electronics*, 13, 1-29.
[31] Li, D., Tian, S., Jin, W., Peng, J., & Duan, M. (2025). Towards a moving target defense based on stochastic games and honeypots. *Information Sciences*, 720, 1-25.
[32] d'Ambrosio, N., Lista, C., Perrone, G., & Romano, S. P. (2025). SMASH: An SDN-MTD framework for efficient honeypot deployment and insider threat mitigation. *Computer Networks*, 269, 1-29.
[33] Alani, M. M. (2024). HoneyTwin: Securing smart cities with machine learning-enabled SDN edge and cloud-based honeypots. *Journal of Parallel and Distributed Computing*, 188, 1-9.
[34] Abdelmaguid, M. A., Hussanein, H. S., & Zulkernine, M. (2025). Securing the unforeseen: Enhancing VANET security with dynamic honeypots and attack rate analysis. *Vehicular Communications*, 55, 1-10.
[35] Alboqmi, R., & Gamble, R. F. (2025). Enhancing Microservice Security Through Vulnerability-Driven Trust in the Service Mesh Architecture. *Sensors*, 25, 1-23.